# Generador de texto con Redes Neuronales Recurrentes

**José Salvador Aguilar Huerta**

Diplomado en Inteligencia Artificial · Módulo 3 · Unidad 5 · Reto 7 · Informe y conclusiones · 1 de noviembre de 2026

## 1. Problema y relevancia

El objetivo fue construir un modelo que, a partir de una **semilla** de texto, continúe la secuencia imitando el estilo discursivo de las conferencias matutinas de la presidenta Claudia Sheinbaum Pardo (septiembre de 2026). Es un problema de **modelado de lenguaje**: estimar la probabilidad del siguiente símbolo dado todo el contexto previo, $P(x_t \mid x_1, \ldots, x_{t-1})$, y generar texto muestreando un símbolo a la vez. Ese mismo principio está detrás de los asistentes conversacionales y del autocompletado. Resolverlo con RNN permite entender cómo una red aprende orden, contexto y dependencias, y por qué después surgieron mecanismos como la atención. El proyecto es educativo y todo texto generado se etiqueta como sintético.

## 2. Datos y limpieza

El corpus tiene 19 conferencias, con 1.63 millones de caracteres. El análisis exploratorio identificó 12 tipos de ruido:

- encabezados y separadores;
- etiquetas de hablante;
- anotaciones del estenógrafo, como *(Inaudible)*;
- corchetes editoriales;
- frases para ceder la palabra, como *"—Adelante, Luisa—."*;
- variantes tipográficas de un mismo signo y caracteres invisibles.

Se implementó un pipeline de 14 pasos con verificaciones automáticas. Solo se conservaron las intervenciones de la Presidenta, para obtener un estilo homogéneo. El resultado fue un corpus de **591 mil caracteres** (≈ 100 mil palabras) y un vocabulario que bajó de 106 a **84 símbolos**. Se conservaron las mayúsculas, los acentos, ¿ ¡ y los números. Para validación se reservaron las **dos últimas conferencias completas** (≈ 8 %), lo que evita la fuga de información que produciría una división aleatoria de ventanas.

## 3. Diseño del modelo

- **Nivel carácter.** Con ~100 mil palabras y ~26 mil palabras distintas (la mayoría con 1–2 apariciones), un modelo por palabras tendría muy pocos ejemplos por clase. A nivel carácter, el vocabulario es pequeño, no hay palabras desconocidas y el modelo aprende la morfología del español.
- **RNN y no CNN.** Las CNN capturan patrones locales de tamaño fijo y son ideales para imágenes. El texto, en cambio, es secuencial, causal y con dependencias de longitud variable. Una RNN mantiene un estado oculto que resume todo lo leído y genera de forma autoregresiva.
- **Arquitectura.** El modelo pasa por un embedding de 128 dimensiones, **dos capas LSTM de 512 unidades** con dropout de 0.3 entre ellas y una capa lineal de salida, con unos 3.5 millones de parámetros. La LSTM resuelve el gradiente que se desvanece de la RNN simple: su estado de celda se actualiza de forma *aditiva* y lo regulan las compuertas de olvido, entrada y salida, lo que le permite recordar contexto lejano. Para validar la elección de forma empírica, se compararon tres modelos con los mismos datos y la misma configuración: **SimpleRNN** (1 × 256, línea base), **GRU** (2 × 512) y **LSTM**.

## 4. Implementación

El modelo se implementó en **PyTorch con CUDA** sobre una GPU RTX 3060. TensorFlow no admite GPU en Windows nativo, lo que motivó el cambio de framework (ADR-013). Se escribió un ciclo de entrenamiento explícito con las siguientes piezas:

- pérdida de entropía cruzada;
- optimizador Adam (lr = $2 \cdot 10^{-3}$);

- recorte de gradiente (norma 5);
- `ReduceLROnPlateau`;
- *early stopping* con paciencia de 5 épocas y checkpoint del mejor modelo.

Las secuencias tienen 100 caracteres y se agrupan en batches de 64. La generación usa muestreo con **temperatura** y pasa el estado oculto de forma explícita de un paso al siguiente. Los tres modelos se entrenaron en **4.9 minutos** en total. Todas las decisiones están registradas en `DECISIONES.md`.

## 5. Resultados y evaluación

| Modelo | Parámetros | val_loss | Perplejidad | BPC | Exactitud | Palabras válidas (T 0.5–0.8) |
|---|---|---|---|---|---|---|
| SimpleRNN | 0.13 M | 1.359 | 3.89 | 1.96 | 60.2 % | 93 % |
| GRU | 2.6 M | 1.196 | 3.31 | 1.73 | 65.1 % | 98 % |
| **LSTM** | 3.5 M | **1.180** | **3.25** | **1.70** | **65.3 %** | **98 %** |

- **La LSTM obtuvo el mejor resultado** y reduce la pérdida un 13 % frente a la SimpleRNN. Con temperatura alta, la SimpleRNN cae a 65 % de palabras válidas, mientras que la LSTM se mantiene en 91 %.
- **El texto generado** usa la puntuación del español, mayúsculas y fórmulas típicas del discurso ("Entonces,", "el pueblo de México").
- **Diversidad y memorización:** con temperatura 0.5–0.8 la diversidad de bigramas está entre 90 % y 97 % y la copia literal del corpus es ~0 %. Con T = 0.2 el texto se vuelve repetitivo, y con T ≥ 1.0 aparecen palabras inventadas.
- **Criterios de aceptación:** se cumplieron 3 de 4. El BPC (1.70) no alcanzó el umbral de 1.6 que se fijó *a priori*. Los modelos con compuertas sobreajustan después de 7 a 10 épocas. Un ajuste con más dropout (0.5) y menos unidades (256) no bajó el piso de BPC ≈ 1.70, lo que indica que **el límite son los datos**, no la capacidad del modelo.

## 6. Conclusiones y aprendizajes

1. **Las compuertas son decisivas.** La diferencia entre la SimpleRNN y la LSTM/GRU confirma en la práctica la teoría del gradiente que se desvanece. La diferencia entre LSTM y GRU (~1 %) es menor: en un corpus pequeño, la GRU es una alternativa eficiente con 25 % menos parámetros.
2. **El modelo aprende el estilo sin saber qué es una palabra.** A partir de caracteres reconstruye la ortografía, la sintaxis local y las muletillas del discurso. Sin embargo, la coherencia solo es local: el tema cambia cada una o dos oraciones y el modelo **inventa cifras y afirmaciones** verosímiles. Por eso es indispensable etiquetar el texto como sintético.
3. **La preparación de datos fue la etapa de mayor impacto.** Una validación honesta, por conferencias completas, da métricas más exigentes pero confiables.
4. **El early stopping fue indispensable** para no quedarse con un modelo sobreajustado. La GPU permitió comparar tres arquitecturas y cuatro variantes en minutos.
5. **Trabajo futuro:** más meses de transcripciones (la mejora con mayor impacto esperado), tokenización por subpalabras y modelos con atención o Transformers para mejorar la coherencia temática.